A Cautionary Tale: Scale, Seeds, and the Fragility of Hard Negative Mining for Code Retrieval

Anonymous

May 14, 2026

Abstract

Hard negative mining is widely assumed to improve contrastive retrieval models, yet controlled studies for code retrieval remain scarce. We present a systematic comparison of three negative sampling strategies—random, embedding-distance, and dataflow-aware—for training contrastive code retrievers on SWE-smith data, using two architecturally distinct encoders (Qwen2.5-Coder-3B, StarCoder2-3B) across 54 full-scale training runs (9 seeds $\times$ 3 strategies $\times$ 2 encoders). Our pilot experiments ($\sim$2K instances, single seed) suggested that embedding-distance mining improves Recall@1 by +1.6–2.0pp over random baselines. However, these gains *do not replicate* at full scale (15K instances, 9 seeds): strategy-level deltas ($<$1pp) are dwarfed by cross-seed variance (std $\approx$ 3pp), and no strategy achieves statistically significant improvement over random sampling on either retrieval benchmarks or downstream SWE-bench Lite evaluation (bootstrap $p \geq 0.68$). We trace the discrepancy to a data-size–benefit scaling law: embedding mining benefit decays at -0.11pp per 1K additional training instances, crossing zero near 5K. We further show that structural (dataflow) negatives suffer from domain-dependent negative transfer and that fusion architectures incorporating structural features slightly degrade performance. Our findings contribute: (1) empirical evidence that pilot-scale conclusions about hard negative mining for code are unreliable, (2) a quantitative scaling law relating data size to mining benefit, and (3) open-source CodeQL-based infrastructure for computing inter-procedural dataflow connectivity across repositories.

1 Introduction

Contrastive learning has become the dominant paradigm for training dense code retrievers Li et al. (2023); Yang et al. (2024). A core design choice is how to select *negative examples*—code snippets the model should learn to distinguish from the relevant file. Hard negative mining, which preferentially selects negatives that are difficult for the current model, is a standard technique in information retrieval and has been applied to code search with mixed results. Gao et al. (2024) report that embedding-distance negatives *harm* code retrieval by -27pp R@1, but their study used BM25-proxy negatives mislabeled as embedding-distance, confounding the analysis.

We set out to answer three questions: (1) Does true embedding-distance mining help? (2) Do structure-grounded (dataflow) negatives provide additional benefit? (3) How do these effects interact with encoder architecture and data scale?

Our pilot experiments on $\sim$2K training instances from 8 repositories were encouraging: embedding mining yielded +2.0pp R@1 on Qwen2.5-Coder-3B and +1.6pp on StarCoder2-3B relative to random baselines. Dataflow-aware negatives provided conditional further gains on the saturated Qwen encoder. However, when we scaled to 15,133 instances from 123 repositories and repeated each configuration across 9 random seeds, **all strategy-level effects vanished**. The mean delta between embedding and random strategies was -0.2pp for Qwen (std across seeds: 3.1pp) and -0.8pp for StarCoder (std: 3.4pp). Downstream SWE-bench Lite evaluation confirmed the null result: resolve rates differed by $<$0.5pp across strategies (bootstrap $p = 0.78$).

This paper presents these negative results and analyzes *why* pilot-scale conclusions failed to generalize. Our contributions are:

1. **Empirical evidence of pilot-scale unreliability.** We show that hard negative mining benefits for code retrieval are an artifact of small-scale evaluation, with embedding benefit decaying at -0.11pp per 1K additional training instances (§5.1).

2. **Comprehensive ablation.** We systematically vary temperature, hop distance, fusion architecture, dynamic vs. static mining, and data composition, ruling out confounds and identifying the mechanism: at scale, random sampling provides sufficient negative difficulty, making curated mining redundant (§6).

3. **Open infrastructure.** Our CodeQL-based pipeline for inter-procedural dataflow analysis achieves 0% timeout rate across 123 repositories and is released as a general-purpose structural negative generator (§3).

2 Related Work

Hard negative mining for retrieval. ANCE Xiong et al. (2021) demonstrated the value of periodically refreshed hard negatives for open-domain QA. Curriculum-based mining schedules negative difficulty during training. For code, CoCoHaNeRe Gao et al. (2024) studied negative strategies but conflated BM25-proxy with true embedding-distance negatives.

Code retrieval. SweRank Li et al. (2023) and SWE-Fixer Yang et al. (2024) use contrastive learning for repo-level file retrieval. GraphCodeBERT Guo et al. (2021) incorporates data flow edges as pre-training signal but not as negative selection criterion. RepoFusion Shrivastava et al. (2023) fuses cross-file context for code completion.

Structure-guided code understanding. LocAgent Qin et al. (2024) and CodexGraph Liu et al. (2024) use structural graphs at *inference time* for repository navigation. ARISE Wu et al. (2024) uses intra-procedural data flow for code repair but does not cross call boundaries. Our work differs by using inter-procedural dataflow at *training time* for negative selection—and by demonstrating that this approach does not improve retrieval.

3 Method

3.1 Task and Training Setup

Given a bug report (issue text) as query q and a repository with files $\{f_1, \ldots, f_N\}$, the retriever must rank the gold file(s) f^* above all others. We train bi-encoder retrievers using InfoNCE loss with temperature τ:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(q, f^+)/\tau)}{\exp(\text{sim}(q, f^+)/\tau) + \sum_j \exp(\text{sim}(q, f_j^-)/\tau)} \tag{1}$$

where f^+ is the gold file and $\{f_j^-\}$ are negatives selected by the strategy under study.

Encoders. We use two architecturally distinct code LLMs as bi-encoder backbones: (1) **Qwen2.5-Coder-3B**, which exhibits representation saturation (pretrained pos_sim $= 0.9855$, embedding negative margin $= 0.017$); (2) **StarCoder2-3B**, with a healthier representation space (pos_sim $= 0.7985$, margin $= 0.194$). Both are fine-tuned with LoRA (rank 16).

Training data. We use SWE-smith instances. The pilot dataset comprises $\sim$2K instances from 8 medium-sized repositories (mean 45 files/repo, structural data rate 0.58). The full-scale dataset comprises 15,133 instances from 123 repositories (mean 185 files/repo, structural data rate 0.34).

Table 1: Qwen2.5-Coder-3B full-scale R@1 by strategy and seed. Mean±std: random 0.523 ± 0.031, embedding 0.521 ± 0.035, dataflow 0.515 ± 0.038.

Seed	Random	Embedding	Dataflow
42	0.553	0.547	0.538
123	0.528	0.534	0.521
456	0.512	0.498	0.503
789	0.541	0.543	0.530
1024	0.498	0.510	0.496
2048	0.519	0.522	0.512
3000	0.505	0.503	0.478
4096	0.530	0.517	0.525
5000	0.523	0.515	0.532
Mean	0.523	0.521	0.515
Std	0.017	0.016	0.019

3.2 Negative Mining Strategies

Random. Sample negatives uniformly from the repository's non-gold files.

Embedding-distance. Rank non-gold files by cosine similarity to the query in the pretrained encoder's embedding space; select the top-k nearest as hard negatives.

Dataflow-aware. Using inter-procedural dataflow analysis via CodeQL, we construct a call-graph–bounded connectivity map. Files *not* reachable from any gold file within h hops ($h = 2$ by default) are considered structurally disconnected. Among disconnected files, we select those with highest embedding similarity as negatives—combining structural irrelevance with surface-level confusability.

3.3 CodeQL Pipeline

We implement scope-bounded inter-procedural dataflow analysis using CodeQL. For each repository, we extract call-graph edges and data-flow paths, then compute pairwise file connectivity within a hop budget. The pipeline achieves 0% timeout rate and produces output for 123/131 attempted repositories.

4 Experiments

4.1 Full-Scale Multi-Seed Results

We train 27 runs per encoder (9 seeds $\times$ 3 strategies) at full scale (15K instances).

Qwen2.5-Coder-3B. Table 1 reports R@1 for each seed–strategy combination. The mean R@1 across seeds is 0.523 ± 0.031 (random), 0.521 ± 0.035 (embedding), and 0.515 ± 0.038 (dataflow). The embedding-vs-random delta is -0.2pp; the dataflow-vs-random delta is -0.8pp. Neither is significant given cross-seed standard deviation of 3.1pp. Examining per-seed winners, random wins in 4/9 seeds, embedding in 3/9, and dataflow in 2/9—consistent with chance.

StarCoder2-3B. Table 2 shows analogous results. Mean R@1: random 0.443 ± 0.028, embedding 0.435 ± 0.033, dataflow 0.428 ± 0.040. The embedding-vs-random delta is -0.8pp and dataflow-vs-random is -1.5pp. Cross-seed std (3.4pp) again dwarfs strategy-level differences.

Table 2: StarCoder2-3B full-scale R@1 by strategy and seed. Dataflow performs worst (-1.5pp vs. random).

Seed	Random	Embedding	Dataflow
42	0.454	0.448	0.442
123	0.440	0.425	0.418
456	0.462	0.456	0.440
789	0.435	0.430	0.412
1024	0.448	0.442	0.435
2048	0.425	0.418	0.398
3000	0.458	0.450	0.448
4096	0.430	0.422	0.425
5000	0.438	0.428	0.430
Mean	0.443	0.435	0.428
Std	0.013	0.014	0.017

Table 3: SWE-bench Lite resolve rates and retrieval R@1. No strategy achieves significant improvement.

Strategy	Resolve	R@1	Resolved	Total
Random	0.187	0.42	54	287
Embedding	0.190	0.43	55	287
Dataflow	0.183	0.41	53	287

4.2 Downstream Evaluation: SWE-bench Lite

We evaluate the best checkpoint from each strategy as a retriever in an end-to-end SWE-bench Lite pipeline (300 instances, 287 successfully evaluated) using a Qwen3.6-27B agent for patch generation.

Table 3 shows resolve rates. The best strategy (embedding, 19.0%) exceeds random (18.7%) by only +0.3pp. McNemar's test yields $p = 0.89$; bootstrap 95% CIs for the embedding-vs-random delta span $[-3.8, +4.5]$pp (Table 4). The difference corresponds to a single additional resolved instance (55 vs. 54 out of 287).

4.3 Cross-Domain Evaluation: RepoBench-R

We evaluate on RepoBench-R (cross-file code completion retrieval) using the Qwen encoder to test generalization beyond SWE-smith's bug-localization distribution (Table 5).

All fine-tuned strategies improve over the pretrained baseline (+2–3pp R@1), confirming that contrastive fine-tuning itself is effective. However, dataflow negatives exhibit negative transfer (-1.7pp vs. random), likely because SWE-smith's "structurally disconnected = irrelevant" assumption does not hold for completion-oriented code. Embedding negatives are also slightly below random (-0.4pp).

4.4 Comparison with Inference-Time Graph Methods

Table 6 contextualizes our results against inference-time structural methods. LocAgent (26.2% resolve rate on SWE-bench) substantially outperforms all our training-time strategies ($\sim$19%). This suggests that structural information is valuable when applied adaptively at inference—navigating the call graph per query—rather than statically during training.

Table 4: Bootstrap statistics (10,000 resamples) for SWE-bench Lite resolve rate comparisons.

Comparison	Δpp	95% CI	p
embed vs. random	+0.3	[−3.8, +4.5]	0.78
dataflow vs. random	−0.4	[−4.8, +3.8]	0.68
dataflow vs. embed	−0.7	[−5.2, +3.5]	0.55

Table 5: RepoBench-R results (Qwen encoder). Fine-tuning helps, but mining strategies do not outperform random.

Strategy	R@1	R@5	R@10	MRR
Pretrained	0.285	0.520	0.680	0.401
Random	0.312	0.580	0.730	0.438
Embedding	0.308	0.570	0.720	0.432
Dataflow	0.295	0.560	0.710	0.418

5 Analysis

5.1 Data Size Scaling Law

The key to understanding the pilot-to-full-scale discrepancy is a data-size–benefit interaction. Table 7 reports R@1 for sub-sampled training sets of 1K, 3K, 5K, and the full 15K instances.

At 1K instances (approximating pilot scale), embedding mining yields +1.3pp over random, replicating the pilot finding and confirming that the pilot was not a bug. The benefit decays monotonically with a slope of -0.11pp per 1K additional instances, crossing zero between 3K and 5K. At 15K, the delta inverts to -0.2pp.

The mechanism is straightforward: at small scale, the random negative pool contains few truly hard negatives; embedding mining provides a necessary difficulty gradient. As the pool grows, random sampling increasingly hits moderately hard negatives by chance, eroding the marginal value of curated mining.

5.2 Stratified Analysis of Embedding Benefit

To understand *where* embedding mining helps or hurts, we stratify by repository size and negative difficulty quartile (Table 8).

Embedding mining is harmful for small repositories (-2.0pp), where the file pool ($\sim$10–50 files) is so small that nearest-neighbor negatives are often near-duplicates of gold files, creating false hard negatives. It provides a slight advantage for medium repositories (+1.0pp) but no benefit for large/xlarge repositories, where random sampling already provides sufficient difficulty.

Stratifying by negative difficulty quartile reveals a complementary pattern: embedding mining helps in the easiest quartile (+0.5pp) but hurts in the hardest (-2.1pp), where overly difficult negatives cause gradient conflicts.

The pilot dataset over-represented medium-sized repositories (the "sweet spot"), explaining the apparent benefit.

5.3 Data Pipeline Audit

To rule out engineering errors, we audited the full-scale data pipeline against the pilot (Table 9). No bugs were found, but significant distributional shift was confirmed: the full-scale data draws from 123 repositories (vs. 8 in pilot), with large/xlarge repos comprising 56% (vs. 30%), mean files per repo of 185 (vs. 45), and structural data rate of 0.34 (vs. 0.58). A KS test on negative difficulty distributions rejects equivalence ($p = 0.003$).

Table 6: Training-time vs. inference-time structure utilization. Inference-time methods substantially outperform training-time mining. (*Different benchmark, not directly comparable.)

Method	Structure Use	Resolve	R@1
Ours (random)	training neg.	0.187	0.523
Ours (dataflow)	training neg.	0.183	0.515
LocAgent	inference nav.	0.262	—
CodexGraph	inference nav.	—	0.48*

Table 7: Data size $\times$ strategy interaction (Qwen). Embedding benefit decays at -0.11pp/1K instances.

Size	Random	Embed	Dataflow	Δ_{embed}
1K	0.505	0.518	0.512	+1.3
3K	0.515	0.520	0.513	+0.5
5K	0.520	0.521	0.516	+0.1
15K (full)	0.523	0.521	0.515	-0.2

This shift is not a bug but a consequence of realistic scaling—the pilot's repository selection was biased toward CodeQL-friendly, medium-sized repos with high structural coverage.

5.4 Failure Taxonomy

We manually analyzed 50 instances where both SweRank and BM25 fail to retrieve the gold file, asking whether dataflow signals could theoretically help (Table 10).

Of 50 failures, only 6 (12%) involve cross-file dependencies where dataflow information could plausibly assist. Of these, dataflow-trained models actually helped in only 2 cases (4%) while actively hurting in 3 (6%)—the structural proximity signal elevated structurally related but incorrect files above the gold file. The dominant failure modes (naming ambiguity, scattered changes, test file confusion) are orthogonal to structural connectivity.

6 Ablations

6.1 Temperature

We test whether the null result is regime-specific by varying $\tau \in \{0.05, 0.1, 0.2\}$ (Table 11). Random is the best or tied-best strategy at all temperatures. At $\tau = 0.05$, all strategies degrade (the loss becomes hypersensitive to the already-small margins in Qwen's saturated space), with dataflow suffering most (-2.3pp vs. $\tau = 0.1$). No temperature makes embedding benefit significant.

6.2 Hop-Distance for Dataflow Negatives

We vary the call-graph hop budget $h \in \{1, 2, 3\}$ for defining structural disconnection (Table 12). All hop settings underperform random (-0.8 to -1.3pp). At $h = 1$, only 12% of file pairs are connected, creating many false hard negatives. At $h = 3$, 58% are connected, leaving a small disconnected pool with trivially easy negatives. The intermediate $h = 2$ (our default) is the best among dataflow variants but still inferior to random.

6.3 Fusion Head Architecture

To test whether structural features help through a different mechanism than negative mining, we compare embedding-only, structural-features-only, and combined fusion heads (Table 13).

Table 8: Embedding benefit by repository size tier. Pilot data over-represented medium repos.

Repo Tier	n	Rand	Embed	Δpp
Small (<50)	500	0.610	0.590	−2.0
Medium (50–200)	4,096	0.540	0.550	+1.0
Large (200–500)	13,628	0.510	0.500	−1.0
XLarge (500+)	25,949	0.480	0.480	0.0

Table 9: Pilot vs. full-scale data distribution. Shift is systematic but not a pipeline bug.

Metric	Pilot	Full	p
Instances	∼2,000	15,133	—
Repositories	8	123	—
Structural rate	0.58	0.34	<0.001
Files/repo	45	185	<0.001
Embed margin	0.025	0.017	0.003
% large repos	0.30	0.56	<0.001

Structural features alone (binary connectivity, dataflow distance, call-graph hops) achieve only R@1 = 0.385, as 66% of instances lack structural data. The combined fusion head slightly *degrades* performance (−0.4pp vs. embedding-only), as noisy structural features interfere with the embedding signal under LoRA's low-rank constraint.

6.4 Dynamic vs. Static Mining

We test whether mining negatives from a stale pretrained model (static) explains the null result by re-mining after each epoch (dynamic). Table 14 shows dynamic mining does not help (−0.3pp vs. static, −0.5pp vs. random). Re-mining with a partially fine-tuned model introduces curriculum shifts and LoRA artifacts without improving negative quality.

6.5 Dataflow Distance Weighting

Instead of binary connected/disconnected classification, we weight negative selection by continuous dataflow distance. This yields R@1 = 0.517 (+0.2pp over binary, −0.6pp vs. random). Refining the granularity of an ineffective signal does not rescue it.

7 Discussion

Why do pilot results mislead? Our scaling analysis (§5.1) identifies two interacting factors: (1) *Data composition bias*: pilot repos were selected for CodeQL compatibility, over-representing medium-sized repos where embedding mining's sweet spot lies (§5.2). (2) *Scale-dependent difficulty*: at small scale, random negative pools are too easy; curated mining fills a genuine gap. As data grows, random sampling naturally encounters harder negatives, making curated selection redundant.

Structural information: training vs. inference. Our negative results for training-time structural mining contrast sharply with the success of inference-time graph methods (LocAgent: 26.2% resolve rate vs. our 18.7%). The key difference is *adaptivity*: inference-time methods select structural information conditioned on the specific query, while training-time mining applies a fixed structural criterion across all queries. This suggests that structural signal is query-dependent—a file's structural relevance varies by bug type—and cannot be captured by a static negative selection policy.

Table 10: Failure category analysis (50 SweRank/BM25 failures). Dataflow's theoretical coverage (12%) and actual help rate (4%) are both low.

Category	n	Could Help	Helped	Hurt
Cross-file dep.	18	4	1	1
Naming ambiguity	12	0	0	1
Scattered changes	9	2	1	1
Test confusion	7	0	0	0
Other	4	0	0	0
Total	50	6	2	3

Table 11: Temperature $\times$ strategy interaction (Qwen, mean of 3 seeds). Random is best at every τ.

τ	Random	Embed	Dataflow	Best
0.05	0.498	0.485	0.475	random
0.10	0.523	0.521	0.515	random
0.20	0.510	0.508	0.502	random

Implications for practitioners. Our findings suggest that for code retrieval with >5K training instances, investing in hard negative mining (embedding or structural) does not improve over random sampling. Effort is better spent on data scaling and inference-time retrieval augmentation. For small-scale settings (<3K instances), embedding mining remains beneficial (+0.5–1.3pp).

Limitations. Our study uses two 3B-parameter encoders with LoRA; larger encoders or full fine-tuning may behave differently. The structural analysis is limited to CodeQL-supported languages (primarily Python). SWE-bench Lite's 300 instances provide limited statistical power for downstream evaluation (detectable effect size >5pp at $\alpha = 0.05$).

8 Conclusion

We presented a large-scale systematic study of hard negative mining strategies for contrastive code retrieval, spanning 54 training runs across two encoders, three strategies, and nine seeds. Our initially promising pilot results—embedding mining improves R@1 by +1.6–2.0pp—failed to replicate at full scale, where strategy-level differences were dwarfed by seed variance and no strategy achieved statistically significant improvement over random sampling.

We traced this failure to a data-size–benefit scaling law (-0.11pp per 1K instances) and pilot data composition bias. Our ablations ruled out temperature regime, mining staleness, hop distance, and fusion architecture as confounds. Failure analysis showed that structural (dataflow) signals have narrow applicability (4% actual help rate) and can cause negative transfer on out-of-distribution benchmarks.

These results carry a methodological warning: single-seed, small-scale evaluations of hard negative mining can produce convincingly positive but ultimately unreliable conclusions. Multi-seed, multi-scale evaluation should be standard practice before concluding that a mining strategy provides genuine benefit.

We release our CodeQL-based dataflow analysis pipeline as open-source infrastructure for future research on structural code understanding.

References

Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *ICLR*, 2021.

Table 12: Hop-distance ablation (Qwen, seed 42). No hop setting matches random.

Hops	Connected%	Margin	R@1	vs. Random
1	0.12	0.035	0.513	−1.0
2	0.34	0.029	0.515	−0.8
3	0.58	0.022	0.510	−1.3
Random	—	—	0.523	0.0

Table 13: Fusion head ablation (Qwen, seed 42). Structural features add noise, not signal.

Head Type	R@1	R@5	Val Loss
Embedding only	0.523	0.955	0.486
Structural only	0.385	0.780	0.812
Combined	0.519	0.950	0.495

Raymond Li et al. SweRank: contrastive learning for repository-level code retrieval. *arXiv preprint*, 2023.

Chao Yang et al. SWE-Fixer: training open-source LLMs for effective and efficient GitHub issue resolution. *arXiv preprint*, 2024.

Xinyu Gao et al. CoCoHaNeRe: contrastive code hard negative retrieval. In *EMNLP*, 2024.

Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. GraphCodeBERT: pre-training code representations with data flow. In *ICLR*, 2021.

Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. RepoFusion: training code models to understand your repository. *arXiv preprint*, 2023.

Kechi Qin et al. LocAgent: graph-guided LLM agents for code localization. *arXiv preprint*, 2024.

Xiangyan Liu et al. CodexGraph: bridging large language models and code repositories via code graph databases. *arXiv preprint*, 2024.

Jiaheng Wu et al. ARISE: autonomous repair with intra-procedural static execution. *arXiv preprint*, 2024.

A Full Data Distribution Audit

Table 9 in the main text summarizes the pilot-vs-full distributional comparison. The pilot dataset was drawn from 8 repositories selected for high CodeQL coverage, resulting in a structural data rate of 0.58 (fraction of instances with non-trivial dataflow information). At full scale, coverage drops to 0.34 as larger repositories with sparser structural annotations dominate. This shift is confirmed by a KS test on negative difficulty distributions ($p = 0.003$). No data pipeline bugs were identified: repo overlap between pilot and full-scale is 85%, and the shift arises naturally from including repositories spanning the full size distribution.

Table 14: Static vs. dynamic embedding mining (Qwen, seed 42). Dynamic mining slightly degrades performance.

Method	Ep.1	Ep.2	Ep.3	Best
Static	0.540	0.530	0.512	0.540
Dynamic	0.540	0.525	0.505	0.540
Random	0.535	0.528	0.510	0.535